16 C Under Windows

- Which Windows…
- Integers
- The Use of typedef
- Pointers in the 32-bit World
  - Memory Management
  - Device Access
- DOS Programming Model
- Windows Programming Model
  - Event Driven Model
- Windows Programming, a Closer Look
- The First Windows Program
- Hungarian Notation
- Summary
- Exercise

So far we have learnt every single keyword, operator and instruction available in C. Thus we are through with the language elements that were there to learn. We did all this learning by compiling our programs using a 16-bit compiler like Turbo C/C++. Now it is time to move on to more serious stuff. To make a beginning one has to take a very important decision—should we attempt to build programs that are targeted towards 16-bit environments like MS-DOS or 32-bit environments like Windows/Linux. Obviously we should choose the 32-bit platform because that is what is in commercial use today and would remain so until 64-bit environment takes over in future. That raises a very important question—is it futile to learn C programming using 16-bit compiler like Turbo C/C++? Absolutely not! The typical 32-bit environment offers so many features that the beginner is likely to feel lost. Contrasted with this, 16-bit compilers offer a very simple environment that a novice can master quickly.

Now that the C fundamentals are out of the way and you are confident about the language features it is time for us to delve into the modern 32-bit operating environments. In today's commercial world 16-bit operating environments like DOS are more or less dead. More and more software is being created for 32-bit environments like Windows and Linux. In this chapter we would explore how C programming is done under Windows. Chapters 20 & 21 are devoted to exploring C under Linux.

Which Windows…

To a common user the differences amongst various versions of Windows like Windows 95,98, ME, NT, 2000, XP, Server 2003 is limited to only visual appearances—things like color of the title bar, shape of the buttons, desktop, task bar, programs menu etc. But the truth is much farther than that. Architecturally there are huge differences amongst them. So many are the differences that Microsoft categorizes the different versions under two major heads—Consumer Windows and Windows NT Family. Windows

95, 98, ME fall under the Consumer Windows, whereas Windows NT, 2000, XP, Server 2003 fall under the Windows NT Family. Consumer Windows was targeted towards the home or small office users, whereas NT family was targeted towards business users. Microsoft no longer provides support for Consumer Windows. Hence in this book we would concentrate only on NT Family Windows. So in the rest of this book whenever I refer to Windows I mean Windows NT family, unless explicitly specified.

Before we start writing C programs under Windows let us first see some of the changes that have happened under Windows environment.

Integers

Under 16-bit environment the size of integer is of 2 bytes. As against this, under 32-bit environment an integer is of 4 bytes. Hence its range is -2147483648 to +2147483647. Thus there is no difference between an **int** and a **long int**. But what if we wish to store the age of a person in an integer? It would be improper to sacrifice a 4-byte integer when we know that the number to be stored in it is hardly going to exceed hundred. In such as case it would be more sensible to use a **short int** since it is only 2 bytes long.

The Use of *typedef*

Take a look at the following declarations:

```
COLORREF color ;
HANDLE h ;
WPARAM w ;
LPARAM l ;
BOOL b ;
```

Are COLORREF, HANDLE, etc. new datatypes that have been added in C under Windows compiler? Not at all. They are merely typedef's of the normal integer datatype.

A typical C under Windows program would contain several such **typedef**s. There are two reasons why Windows-based C programs heavily make use of **typedef**s. These are:

(a) A typical Windows program is required to perform several complex tasks. For example a program may print documents, send mails, perform file I/O, manage multiple threads of execution, draw in a window, play sound files, perform operations over the network apart from normal data processing tasks. Naturally a program that carries out so many tasks would be very big in size. In such a program if we start using the normal integer data type to represent variables that hold different entities we would soon lose track of what that integer value actually represents. This can be overcome by suitably typedefining the integer as shown above.

(b) At several places in Windows programming we are required to gather and work with dissimilar but inter-related data. This can be done using a structure. But when we define any structure variable we are required to precede it with the keyword **struct**. This can be avoided by using **typedef** as shown below:

```
struct rect
{
    int top ;
    int left ;
    int right ;
    int bottom ;
} ;

typedef struct rect RECT ;
typedef struct rect* PRECT ;
```

```
RECT r ;
PRECT pr ;
```

What have we achieved out of this? It makes user-defined data types like structures look, act and behave similar to standard data types like integers, floats, etc. You would agree that the following declarations

```
RECT r ;
int i ;
```

are more logical than

```
struct RECT r ;
int i ;
```

Imagine a situation where each programmer **typedef**s the integer to represent a color in different ways. Some of these could be as follows:

```
typedef int COL ;
typedef int COLOR ;
typedef int COLOUR ;
typedef int COLORREF ;
```

To avoid this chaos Microsoft has done several **typedef**s for commonly required entities in Windows programming. All these have been stored in header files. These header files are provided as part of 32-bit compiler like Visual C++.

Pointers in the 32-bit World

In a 16-bit world (like MS-DOS) we could run only one application at a time. If we were to run another program we were required to terminate the first one before launching the second. As only one program (task) could run at a time this environment was

called single-tasking environment. Since only one program could run at any given time entire resources of the machine like memory and hardware devices were accessible to this program. Under 32-bit environment like Windows several programs reside and work in memory at the same time. Hence it is known as a multi-tasking environment. But the moment there are multiple programs running in memory there is a possibility of conflict if two programs simultaneously access the machine resources. To prevent this, Windows does not permit any application direct access to any machine resource. To channelize the access without resulting into conflict between applications several new mechanisms were created in the Microprocessor & OS. This had a direct bearing on the way the application programs are created. This is not a Windows OS book. So we would restrict our discussion about the new mechanisms that have been introduced in Windows to topics that are related, to C programming. These topics are 'Memory Management and Device Access'.

Memory Management

Since users have become more demanding, modern day applications have to contend with these demands and provide several features in them. To add to this, under Windows several such applications run in memory simultaneously. The maximum allowable memory—1 MB—that was used in 16-bit environment was just too small for this. Hence Windows had to evolve a new memory management model. Since Windows runs on 32-bit microprocessors each CPU register is 32-bit long. Whenever we store a value at a memory location the address of this memory location has to be stored in the CPU register at some point in time. Thus a 32-bit address can be stored in these registers. This means that we can store 2^{32} unique addresses in the registers at different times. As a result, we can access 4 GB of memory locations using 32-bit registers. As pointers store addresses, every pointer under 32-bit environment also became a 4-byte entity.

However, if we decide to install 4 GB memory it would cost a lot. Hence Windows uses a memory model which makes use of as much of physical memory (say 128 MB) as has been installed and simulates the balance amount of memory (4 GB – 128 MB) on the hard disk. Be aware that this balance memory is simulated as and when the need to do so arises. Thus memory management is demand based.

Note that programs cannot execute straight-away from hard disk. They have to be first brought into physical memory before they can get executed. Suppose there are multiple programs already in memory and a new program starts executing. If this new program needs more memory than what is available right now, then some of the existing programs (or their parts) would be transferred to the disk in order to free the physical memory to accommodate the new program. This operation is often called page-out operation. Here page stands for a block of memory (usually of size 4096 bytes). When that part of the program that was paged out is needed it is brought back into memory (called page-in operation) and some other programs (or their parts) are paged out. This keeps on happening without a common user's knowledge all the time while working with Windows. A few more facts that you must note about paging are as follows:

(a) Part of the program that is currently executing might also be paged out to the disk.

(b) When the program is paged in (from disk to memory) there is no guarantee that it would be brought back to the same physical location where it was before it was paged out.

Now imagine how the paging operations would affect our programming. Suppose we have a pointer pointing to some data present in a page. If this page gets paged out and is later paged in to a different physical location then the pointer would obviously have a wrong address. Hence under Windows the pointer never holds the physical address of any memory location. It always holds a virtual address of that location. What is this virtual address? At

its name suggests it is certainly not a real address. It is a number, which contains three parts. These parts when used in conjunction with a CPU register called CR3 and contents of two tables called Page Directory Table and Page Table leads to the actual physical address. This is shown in Figure 16.1.

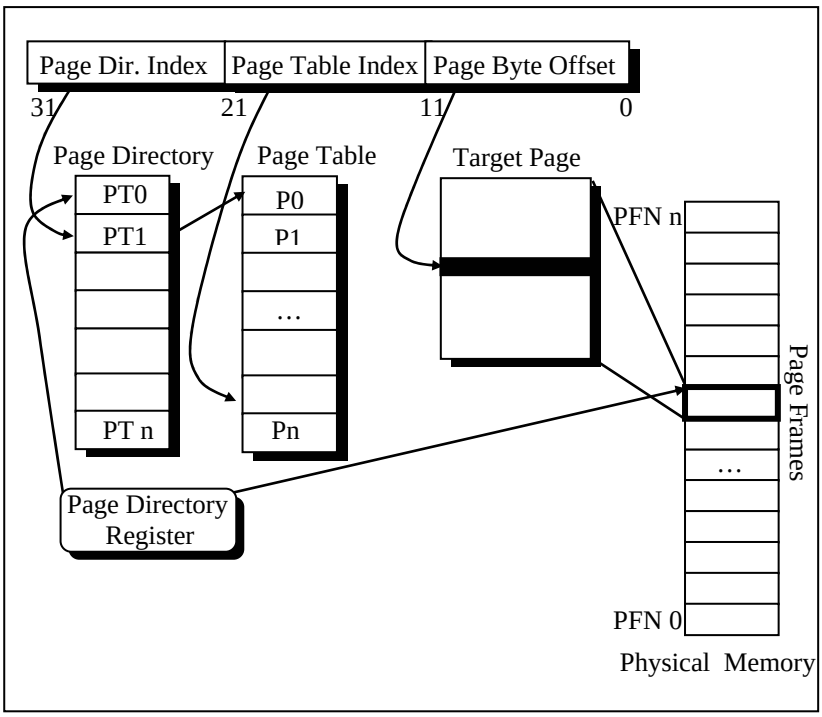

Figure 16.1

The CR3 register holds the physical location of Page Directory Table. The left part of the 32-bit virtual address holds the index into the Page Directory Table. The value present at this index is the starting address of the Page Table. The middle part of the 32-bit virtual address holds the index into the Page Table. The value present at this index is the starting address of the physical page in memory. The right part of the 32-bit virtual address holds the byte

offset (from the start of the page) of the physical memory location to be accessed.

Note that the CR3 register is not accessible from an application. Hence an application can never directly reach a physical address. Also, as the paging activity is going on the OS would suitably keep updating the values in the two tables.

Device Access

All devices under Windows are shared amongst all the running programs. Hence no program is permitted a direct access to any of the devices. The access to a device is routed through a device driver program, which finally accesses the device. There is a standard way in which an application can communicate with the device driver. It is device driver's responsibility to ensure that multiple requests coming from different applications are handled without causing any conflict. This standard way of communication is discussed in detail in Chapter 17.

DOS Programming Model

Typical 16-bit environments like DOS use a sequential programming model. In this model programs are executed from top to bottom in an orderly fashion. The path along which the control flows from start to finish may vary during each execution depending on the input that the program receives or the conditions under which it is run. However, the path remains fairly predictable. C programs written in this model begin execution with **main( )** (often called entry point) and then call other functions present in the program. If you assume some input data you can easily walk through the program from beginning to end. In this programming model it is the program and not the operating system that determines which function gets called and when. The operating system simply loads and executes the program and then waits for it to finish. If the program wishes it can take help of the OS to carry

out jobs like console I/O, file I/O, printing, etc. For other operations like generating graphics, carrying out serial communication, etc. the program has to call another set of functions called ROM-BIOS functions.

Unfortunately the DOS functions and the BIOS functions do not have any names. Hence to call them the program had to use a mechanism called interrupts. This is a messy affair since the programmer has to remember interrupt numbers for calling different functions. Moreover, communication with these functions has to be done using CPU registers. This lead to lot of difficulties since different functions use different registers for communication. To an extent these difficulties are reduced by providing library functions that in turn call the DOS/BIOS functions using interrupts. But the library doesn't have a parallel function for every DOS/BIOS function. DOS functions either call BIOS functions or directly access the hardware.

At times the programs are needed to directly interact with the hardware. This has to be done because either there are no DOS/BIOS functions to do this, or if they are there their reach is limited.

Figure 16.2 captures the essence of the DOS programming model.

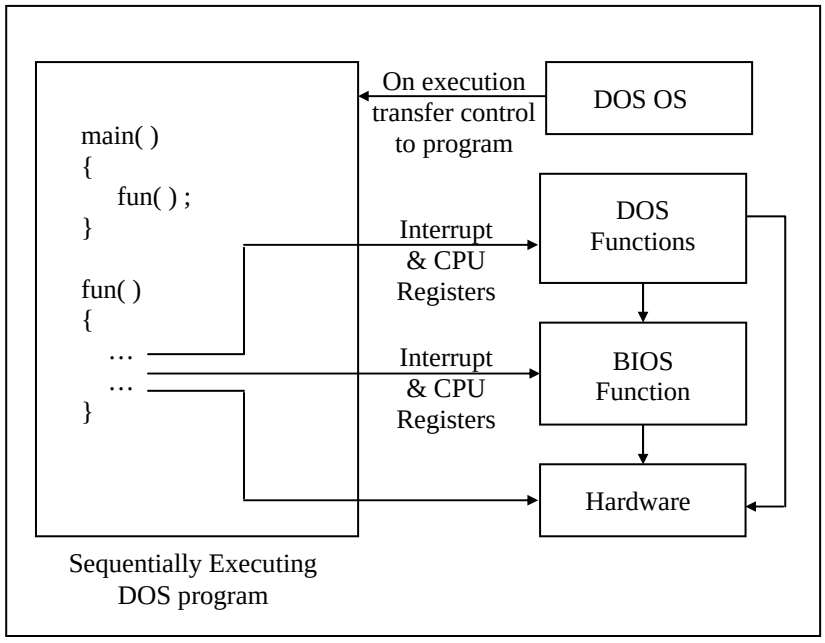

Figure 16.2

From the above discussion you can gather that there are several limitations in the DOS programming model. These have been listed below:

No True Reuse

The library functions that are called from each program become part of the executable file (.EXE) for that program. Thus the same functions get replicated in several EXE files, thereby wasting precious disk space.

Inconsistent Look and Feel

Every DOS program has a different user interface that the user has to get used to before he can start getting work out of the program. For example, successful DOS-based software like Lotus 1-2-3, Foxpro, Wordstar offered different types of menus. This happened because DOS/BIOS doesn't provide any functions for creating user interface elements like menus. As the look and feel of all DOS based programs is different, the user takes a lot of time in learning how to interact with the program

Messy Calling Mechanism

It is difficult to remember interrupt numbers and the registers that are to be used for communication with DOS/BIOS functions. For example, if we are to position the cursor on the screen using a BIOS function we are required to remember the following details:

Interrupt number – 16
CPU Registers to be used:
    AH – 2 (service number)
    DH – Row number where cursor is to be positioned
    DL – Column number where cursor is to be positioned

While using these interrupt numbers and registers there is always a chance of error.

Hardware Dependency

DOS programs are always required to bother about the details of the hardware on which they are running. This is because for every new piece of hardware introduced there are new interrupt numbers and new register details. Hence DOS programmers are under the constant fear that if the hardware on which the programs are running changes then the program may crash.

Moreover the DOS programmer has to write lot of code to detect the hardware on which his program is running and suitably make use of the relevant interrupts and registers. Not only does this make the program lengthy, the programmer has to understand a lot of technical details of the hardware. As a result the programmer has to spend more time in understanding the hardware than in the actual application programming.

Windows Programming Model

From the perspective of the user the shift from MS-DOS to Windows OS involves switching over to a Graphical User Interface from the typical Text Interface that MS-DOS offers. Another change that the user may feel and appreciate is the ability of Windows OS to execute several programs simultaneously, switching effortlessly from one to another by pointing at windows and clicking them with the mouse. Mastering this new GUI environment and getting comfortable with the multitasking feature is at the most a matter of a week or so. However, from the programmer's point of view programming for Windows is a whole new ball game!

Windows programming model is designed with a view to:

(a) Eliminate the messy calling mechanism of DOS
(b) Permit true reuse of commonly used functions
(c) Provide consistent look and feel for all applications
(d) Eliminate hardware dependency

Let us discuss how Windows programming model achieves this.

Better Calling Mechanism

Instead of calling functions using Interrupt numbers and registers Windows provides functions within itself which can be called using names. These functions are called API (Application Programming Interface) functions. There are literally hundreds of

API functions available. They help an application to perform various tasks such as creating a window, drawing a line, performing file input/output, etc.

True Reuse

A C under Windows program calls several API functions during course of its execution. Imagine how much disk space would have been wasted had each of these functions become part of the EXE file of each program. To avoid this, the API functions are stored in special files that have an extension .DLL.

DLL stands for Dynamic Link Libraries. A DLL is a binary file that provides a library of functions. The functions present in DLLs can be linked during execution. These functions can also be shared between several applications running in Windows. Since linking is done dynamically the functions do not become part of the executable file. As a result, the size of EXE files does not go out of hand. It is also possible to create your own DLLs. You would like to do this for two reasons:

(a) Sharing common code between different executable files.

(b) Breaking an application into component parts to provide a way to easily upgrade application's components.

The Windows API functions come in three DLL files. Figure 16.3 lists these filenames along with purpose of each.

DLL	Description
USER32.DLL	Contains functions that are responsible for window management, including menus, cursors, communications, timer etc.
GDI32.DLL	Contains functions for graphics drawing and painting
KERNEL32.DLL	Contains functions to handle memory management, threading, etc.

Figure 16.3

Consistent Look and Feel

Consistent look and feel means that each program offers a consistent and similar user interface. As a result, user doesn't have to spend long periods of time mastering a new program. Every program occupies a window—a rectangular area on the screen. A window is identified by a title bar. Most program functions are initiated through the program's menu. The display of information too large to fit on a single screen can be viewed using scroll bars. Some menu items invoke dialog boxes, into which the user enters additional information. One dialog box is found in almost every Windows program. It opens a file. This dialog box looks the same (or very similar) in many different Windows programs, and it is almost always invoked from the same menu option.

Once you know how to use one Windows program, you're in a good position to easily learn another. The menus and dialog boxes allow user to experiment with a new program and explore its features. Most Windows programs have both a keyboard interface and a mouse interface. Although most functions of Windows programs can be controlled through the keyboard, using the mouse is often easier for many chores.

From the programmer's perspective, the consistent user interface results from using the Windows API functions for constructing menus and dialog boxes. All menus have the same keyboard and mouse interfaces because Windows—rather than the application program—handles this job.

Hardware Independent Programming

As we saw earlier a Windows program can always call Windows API functions. Thus an application can easily communicate with OS. What is new in Windows is that the OS can also communicate with application. Let us understand why it does so with the help of an example.

Suppose we have written a program that contains a menu item, which on selection is supposed to display a string "Hello World" in the window. The menu item can be selected either using the keyboard or using the mouse. On executing this program it will perform the initializations and then wait for the user input. Sooner or later the user would press the key or click the mouse to select the menu-item. This key-press or mouse-click is known as an 'event'. The occurrence of this event is sensed by the keyboard or mouse device driver. The device driver would now inform Windows about it. Windows would in turn notify the application about the occurrence of this event. This notification is known as a 'message'. Thus the OS has communicated with the application. When the application receives the message it communicates back with the OS by calling a Windows API function to display the string "Hello World" in the window. This API function in turn communicates with the device driver of the graphics card (that drives the screen) to display the string. Thus there is a two-way communication between the OS and the application. This is shown in Figure 16.4.

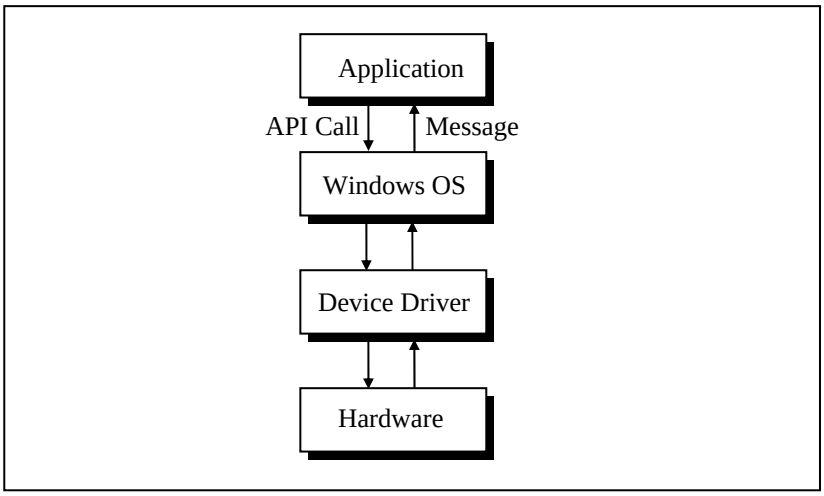

Figure 16.4

Suppose the keyboard and the mouse are now replaced with a new keyboard and mouse. Doing so would not affect the application at all. This is because at no time does the application carry out any direct communication with the devices. Any differences that may be there in the new set of mouse and keyboard would be handled the device driver and not by the application program. Similarly, if the screen or the graphics card is replaced no change would be required in the program. In short hardware independence at work! At times a change of device may necessitate a change in the device driver program, but never a change in the application.

Event Driven Model

When a user interacts with a Windows program a lot of events occur. For each event a message is sent to the program and the program reacts to it. Since the order in which the user would interact with the user-interface elements of the program cannot be predicted the order of occurrence of events, and hence the order of messages, also becomes unpredictable. As a result, the order of

calling the functions in the program (that react to different messages) is dictated by the order of occurrence of events. Hence this programming model is called 'Event Driven Programming Model'.

That's really all that is there to event-driven programming. Your job is to anticipate what users are likely to do with your application's user interface objects and have a function waiting, ready to execute at the appropriate time. Just when that time is, no one except the user can really say.

Windows Programming, a Closer Look

There can be hundreds of ways in which the user may interact with an application. In addition to this some events may occur without any user interaction. For example, events occur when we create a window, when the window's contents are to be drawn, etc. Not only this, occurrence of one event may trigger a few more events. Thus literally hundreds of messages may be sent to an application thereby creating a chaos. Naturally, a question comes—in which order would these messages get processed by the application. Order is brought to this chaos by putting all the messages that reach the application into a 'Queue'. The messages in the queue are processed in First In First Out (FIFO) order.

In fact the OS maintains several such queues. There is one queue, which is common for all applications. This queue is known as 'System Message Queue'. In addition there is one queue per application. Such queues are called 'Application Message Queues'. Let us understand the need for maintaining so many queues.

When we click a mouse and an event occurs the device driver posts a message into the System Message Queue. The OS retrieves this message finds out with regard to which application the message has been sent. Next it posts a message into the

Application Message Queue of the application in which the mouse was clicked. Refer Figure 16.5.

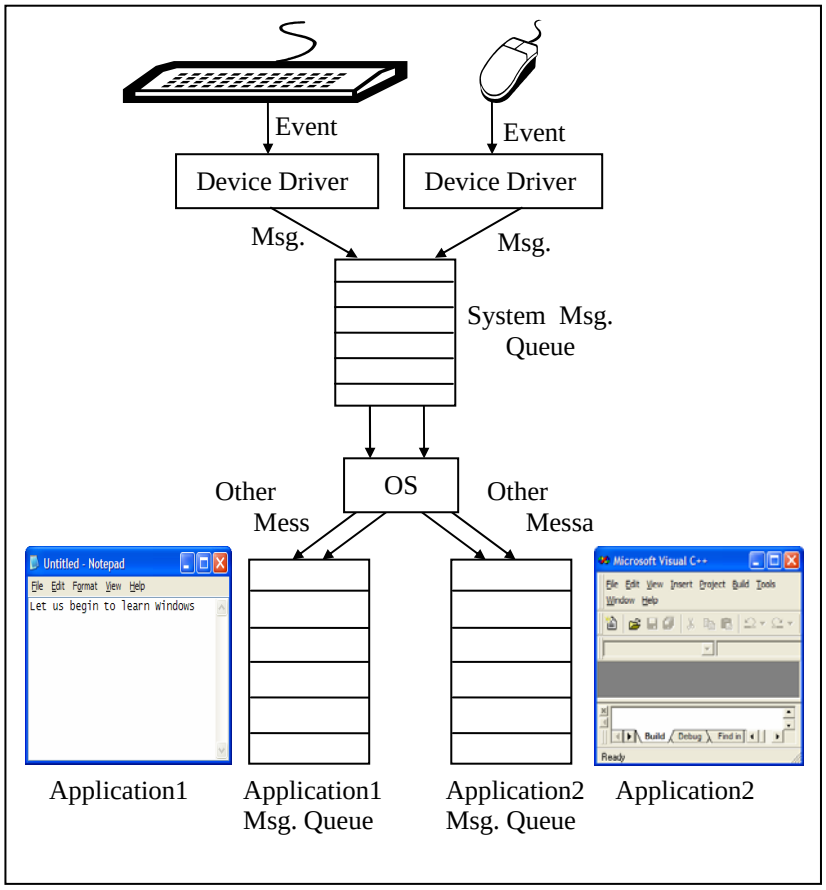

Figure 16.5

I think now we have covered enough ground to be able to actually start C under Windows programming. Here we go…

The First Windows Program

To keep things simple we would begin with a program that merely displays a "Hello" message in a message box. Here is the program…

```
#include <windows.h>
int _stdcall WinMain ( HINSTANCE hInstance, HINSTANCE hPrevInstance,
                       LPSTR lpszCmdline, int nCmdShow )
{
    MessageBox ( 0, "Hello!", "Title", 0 ) ;
    return ( 0 ) ;
}
```

Naturally a question would come to your mind—how do I create and run this program and what output does it produce. Firstly take a look at the output that it produces. Here it is…

Figure 16.6

Let us now look at the steps that one needs to carry to create and execute this program:

(a) Start VC++ from 'Start | Programs | Microsoft Visual C++ 6.0'. The VC++ IDE window will get displayed.
(b) From the File | New menu, select 'Win32 Application', and give a project name, say, 'sample1'. Click on OK.
(c) From the File | New menu, select 'C++ Source File', and give a suitable file name, say, 'sample1'. Click on OK.
(d) The 'Win32 Application-Step 1 of 1' window will appear. Select 'An empty project' option and click 'Finish' button.

(e) A 'New Project Information' dialog will appear. Close it by clicking on OK.
(f) Again select 'File | New | C++ Source File'. Give the file name as 'sample1.c'. Click on OK.
(g) Type the program in the 'sample1.c' file that gets opened in the VC++ IDE.
(h) Save this file using 'Save' option from the File menu.

To execute the program follow the steps mentioned below:

(a) From the Build menu, select 'Build sample1.exe'.
(b) Assuming that no errors were reported in the program, select 'Execute sample1.exe' from the Build menu.

Let us now try to understand the program. The way every C under DOS program begins its execution with **main( )**, every C under Windows program begins its execution with **WinMain( )**. Thus **WinMain( )** becomes the entry point for a Windows program. A typical **WinMain( )** looks like this:

```
int __stdcall WinMain ( HINSTANCE hInstance, HINSTANCE hPrevInstance,
                        LPSTR  lpszCmdLine, int nCmdShow )
```

Note the **__stdcall** before **WinMain( )**. It indicates the calling convention used by the **WinMain( )** function. Calling Conventions indicate two things:

(a) The order (left to right or right to left) in which the arguments are pushed onto the stack when a function call is made.

(b) Whether the caller function or called function removes the arguments from the stack at the end of the call.

Out of the different calling conventions available most commonly used conventions are **__cdecl** and **__stdcall** . Both these calling conventions pass arguments to functions from right to left. In **__cdecl** the stack is cleaned up by the calling function, whereas in case of **__stdcall** the stack is cleaned up by the called function. All

API functions use **__stdcall** calling convention. If not mentioned, **__cdecl** calling convention is assumed by the compiler.

HINSTANCE and LPSTR are nothing but **typedef**s. The first is an **unsigned int** and the second is a **pointer** to a **char**. These macros are defined in 'windows.h'. This header file must always be included while writing a C program under Windows. **hInstance**, **hPrevInstance**, **lpszCmdLine** and **nCmdShow** are simple variable names. In place of these we can use **i**, **j**, **k** and **l** respectively. Let us now understand the meaning of these parameters as well as the rest of the program.

– **WinMain( )** receives four parameters which are as under:

hInstance: This is the 'instance handle' for the running application. Windows creates this ID number when the application starts. We will use this value in many Windows functions to identify an application's data.

A handle is simply a 32-bit number that refers to an entity. The entity could be an application, a window, an icon, a brush, a cursor, a bitmap, a file, a device or any such entity. The actual value of the handle is unimportant to your programs, but the Windows module that gives your program the handle knows how to use it to refer to an entity. What is important is that there is a unique handle for each entity and we can refer and reach the entity only using its handle.

hPrevInstance: This parameter is a remnant of earlier versions of Windows and is no longer significant. Now it always contains a value 0. It is being persisted with only to ensure backward compatibility.

lpszCmdLine: This is a pointer to a character string containing the command line arguments passed to the program. This is similar to the **argv**, **argc** parameters passed to **main( )** in a DOS program.

nCmdShow: This is an integer value that is passed to the function. This integer tells the program whether the window that it creates should appear minimized, as an icon, normal, or maximized when it is displayed for the first time.

- The **MessageBox( )** function pops up a message box whose title is 'Title' and which contains a message 'Hello!'.
- Returning **0** from **WinMain( )** indicates success, whereas, returning a nonzero value indicates failure.
- Instead of printing 'Hello!' in the message box we can print the command line arguments that the user may supply while executing the program. The command line arguments can be supplied to the program by executing it from Start | Run as shown in Figure 16.7.

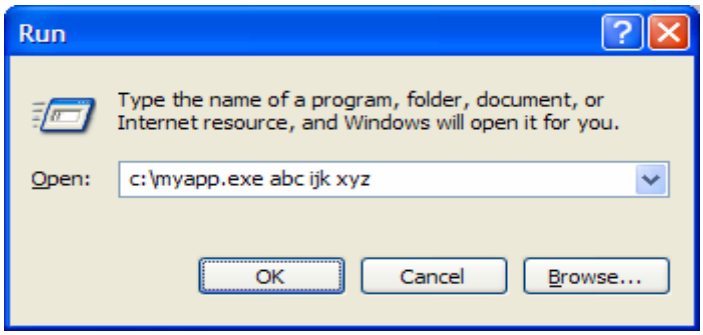

Figure 16.7

Note from Figure 16.7 that 'myapp.exe' is the name of our application, whereas, 'abc ijk xyz' represents command line arguments. The parameter **lpszCmdline** points to the string "abc ijk xyz". This string can be printed using the following statement:

```
MessageBox ( 0, lpszCmdline, "Title", 0 ) ;
```

If the entire command line including the filename is to be retrieved we can use the **GetCommandLine( )** function.

Hungarian Notation

Hungarian Notation is a variable-naming convention so called in the honor of the legendary Microsoft programmer Charles Simonyi. According to this convention the variable name begins with a lower case letter or letters that denotes the data type of the variable. For example, the **sz** prefix in **szCmdLine** stands for 'string terminated by zero'; the prefix **h** in **hInstance** stands for 'handle'; the prefix **n** in **nCmdShow** stands for **int**. Prefixes are often combined to form other prefixes, as **lpsz** in **lpszCmdLine** stands for 'long pointer to a zero terminated string'. Though basically this notation is a good idea nowadays its usage is discouraged. This is because when a transition happens from say a 16-bit code to 32-bit code then a whole lot of variable names have to be changed. For example, suppose the 16-bit code used 2-byte and 4-byte integer variables called **wParam** and **lParam**, where **w** indicated a 16-bit integer (word) and a 32-bit integer (long) respectively. When this code is ported to a 32-bit environment **wParam** had to be changed to **lParam** since in this environment every integer is 4 bytes long. You would agree that if we follow the Hungarian notation then we would have to make a whole lot of changes in the variable names when we port the code to a 32-bit or a 64-bit environment. Hence the usage of this convention is nowadays discouraged.

Summary

(a) Under Windows an integer is four bytes long. To use a two-byte integer pre-qualify it with **short**.
(b) Under Windows a pointer is four bytes long.
(c) Windows programming involves a heavy usage of **typedef**s.
(d) DOS uses a Sequential Programming Model, whereas, Windows uses an Event Driven Programming Model.
(e) Entry point of every Windows program is a function called **WinMain( )**.

(f) Windows does not permit direct access to memory or hardware devices.
(g) Windows uses a Demand-based Virtual Memory Model to manage memory.
(h) Under Windows there is two-way communication between the program and the OS.
(i) Windows maintains a system message queue common for all applications.
(j) Windows maintains an application message queue per running application.
(k) Calling convention decides the order in which the parameters are passed to a function and whether the calling function or the called function clears the stack.
(l) Commonly used calling conventions are **__cdecl** and **__stdcall**.
(m) Hungarian notation though good its usage is not recommended any more.

Exercise

[A] State True or False:

(a) MS-DOS uses a procedural programming model.
(b) A Windows program can directly call a device driver program for a device.
(c) API functions under Windows do not have names.
(d) DOS functions are called using an interrupt mechanism.
(e) Windows uses a 4 GB virtual memory space.
(f) Size of a pointer under Windows depends upon whether it is **near** or **far**.
(g) Under Windows the address stored in a pointer is a virtual address and not a physical address.
(h) One of the parameters of **WinMain( )** called **hPrevInstance** is no longer relevant.

[B] Answer the following:

(a) Why is Event-driven Programming Model better than the Sequential Programming Model?
(b) What is the meaning of different parts of the address stored in a pointer under Windows environment?
(c) Why Windows does not permit direct access to hardware?
(d) What is the difference between an event and a message?
(e) Why Windows maintains a different message queue for each application?
(f) In which different situations messages get posted into an application message queue?

[C] Attempt the following:

(a) Write a program that prints the value of **hInstance** in a message box.
(b) Write a program that displays three buttons 'Yes', 'No' 'Cancel' in the message box.
(c) Write a program that receives a number as a command line argument and prints its factorial value in a message box.
(d) Write a program that displays command line arguments including file name in a message box.

17 Windows Programming

- The Role of a Message Box
- Here comes the window…
- More Windows
- A Real-World Window
  - Creation and Displaying of Window
  - Interaction with Window
  - Reacting to Messages
- Program Instances
- Summary
- Exercise

Event driven programming requires a change in mind set. I hope Chapter 16 has been able to bring about this change. However this change would be bolstered by writing event driven programs. This is what this chapter intends to do. I am hopeful that by the time you reach the end of this chapter you would be so comfortable with it as if you have been using it all your life.

The Role of a Message Box

Often we are required to display certain results on the screen during the course of execution of a program. We do this to ascertain whether we are getting the results as per our expectations. In a sequential DOS based program we can easily achieve this using **printf( )** statements. Under Windows screen is a shared resource. So you can imagine what chaos would it create if all running applications are permitted to write to the screen. You would not be able to make out which output is of what application. Hence no Windows program is permitted to write anything directly to the screen. That's where a message box enters the scene. Using it we can display intermediate results during the course of execution of a program. It can be dismissed either by clicking the 'close button' in its title bar or by clicking the OK button present in it. There are numerous variations that you can try with the **MessageBox( )**. Some of these are given below

```
MessageBox ( 0, "Are you sure", "Caption", MB_YESNO ) ;
MessageBox ( 0, "Print to the Printer", "Caption", MB_YESNO CANCEL) ;
MessageBox ( 0, "icon is all about style", "Caption", MB_OK |
                    MB_ICONINFORMATION ) ;
```

You can put the above statements within **WinMain( )** and see the results for yourself. Though the above message boxes give you flexibility in displaying results, button, icons, there is a limit to which you can stretch them. What if we want to draw a free hand drawing or display an image, etc. in the message box. This would

not be possible. To achieve this we need to create a full-fledged window. The next section discusses how this can be done.

Here Comes the window…

Before we proceed with the actual creation of a window it would be a good idea to identify the various elements of it. These are shown in Figure 17.1.

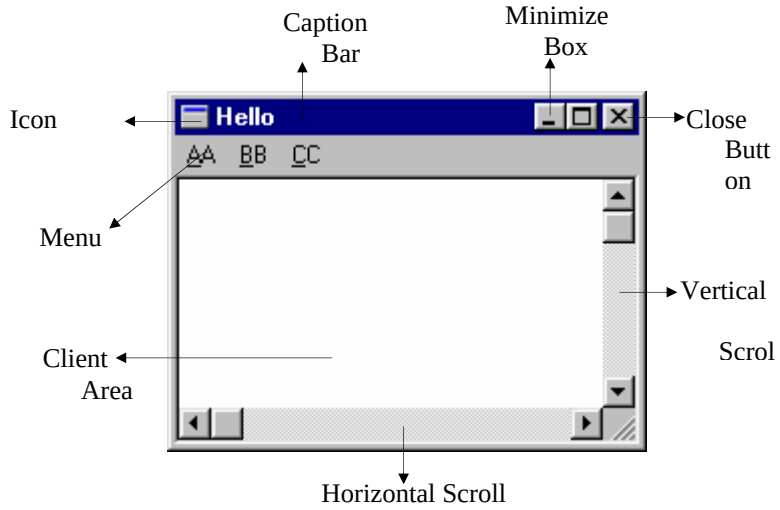

Figure 17.1

Note that every window drawn on the screen need not necessarily have every element shown in the above figure. For example, a window may not contain the minimize box, the maximize box, the scroll bars and the menu.

Let us now create a simple program that creates a window on the screen. Here is the program…

```
#include <windows.h>
```

```
int _stdcall WinMain ( HINSTANCE hInstance, HINSTANCE hPrevInstance,
                       LPSTR lpszCmdLine, int nCmdShow )
{
    HWND h ;

    h = CreateWindow ( "BUTTON", "Hit Me", WS_OVERLAPPEDWINDOW,
                       10, 10, 150, 100, 0, 0, i, 0 ) ;
    ShowWindow ( h, nCmdShow ) ;
    MessageBox ( 0, "Hi!", "Waiting", MB_OK ) ;
    return 0 ;
}
```

Here is the output of the program…

Figure 17.2

Let us now understand the program. Every window enjoys certain properties—background color, shape of cursor, shape of icon, etc. All these properties taken together are known as 'window class'. The meaning of 'class' here is 'type'. Windows insists that a window class should be registered with it before we attempt to create windows of that type. Once a window class is registered we can create several windows of that type. Each of these windows would enjoy the same properties that have been registered through the window class. There are several predefined window classes. Some of these are BUTTON, EDIT, LISTBOX, etc. Our program has created one such window using the predefined BUTTON class.